2. Google Colab — Load the data

```
!pip -q install pyspark
```

```
from google.colab import filesfrom pyspark.sql import SparkSessionfrom py
```

3. Exercises — Lazy Execution in PySpark

Complete these exercises without immediately calling an action after every transformation.

Part A — Basic transformations and actions

1. Filter all transactions where `quantity > 5`. Store the result in a new DataFrame without displaying it.
2. Select only `order_id`, `product_id`, `quantity`, and `unit_price`. Use `show()` to trigger execution.
3. Filter transactions with `unit_price > 20000`. Use `count()` to determine the number of matching records.
4. Create a new column `gross_amount` by multiplying `quantity * unit_price`. Inspect the execution plan using `explain()` before displaying the results.
5. Filter transactions with `order_status = "Completed"` and sort them by `unit_price` descending. Trigger execution using `take(10)`.

Part B — Chained transformations

6. Create a pipeline that filters completed orders, calculates gross amount, selects five relevant columns, and sorts by gross amount descending. Use only one action at the end.
7. Filter orders with discounts of at least 10%, calculate the discounted sales amount, and display the top 15 orders.
8. Group transactions by `payment_method`, calculate total quantity sold, and sort the results in descending order.
9. Group transactions by `sales_channel` and `order_status`, then calculate the transaction count for each combination.
10. Create a DataFrame containing only `order_id`, `store_id`, `product_id`, and `quantity`. Use `drop()` to remove `store_id`, then use `first()` to retrieve one record.

Part C — Joins and aggregations

11. Join `sales_df` with `products_df` using `product_id`. Select product name, category, quantity, and unit price. Display 20 records.
12. Join `sales_df` with `stores_df` using `store_id`. Group by city and calculate the total quantity sold.
13. Join all three DataFrames. Calculate total completed-order revenue for each product category and region.
14. Filter completed transactions, join the product master, calculate discounted revenue, and display the top five products by total revenue.

Part D — Understanding lazy execution

15. Create five consecutive transformations using `filter()`, `select()`, `withColumn()`, `groupBy()`, and `orderBy()`. Add a Python `print()` after each transformation. Observe when Spark actually processes the data.
16. Create a filtered DataFrame and inspect it with `explain()`. Then call `show()` and `count()` separately. Observe the Spark jobs in the Colab output.
17. Use `collect()` to retrieve the grouped total quantity for each product category. Explain why collecting a small aggregated result is safer than collecting all 25,000 rows.
18. Filter completed orders, calculate discounted revenue, and save the resulting DataFrame as Parquet using `write.mode("overwrite").parquet()`. Identify which operation triggers execution.